MAD-2: Placement Portal Application Report

1. Student Details

- **Name:** Sibani Tiwari
- **Roll Number:** 23f3003311
- **Email:** 23f3003311@ds.study.iitm.ac.in

- **About Me:** I am a student at the IIT Madras BS Degree program with a strong interest in full-stack web development and backend systems. I enjoy building functional, real-world applications that connect multiple user roles and automate business processes.

2. Project Details

- **Project Title:** Placement Portal Application (PPA)
- **Problem Statement:** To design and build a centralized web-based application that streamlines the campus placement process. It allows companies to post job drives, students to apply based on eligibility, and administrators to moderate the entire ecosystem while generating automated reports.
- **Approach:** The app was built as a decoupled monolith using Flask for the backend REST APIs and Vue.js (via CDN) for a dynamic, reactive frontend. SQLite is used for persistent data storage, while Redis and Celery manage asynchronous background tasks like sending daily reminder emails and generating monthly PDF reports.

3. AI/LLM Declaration

I used Gemini to assist in writing boilerplate SQLAlchemy model definitions, formatting the HTML templates with Bootstrap 5, and debugging Celery worker configurations.

- The extent of AI/LLM usage is around 10–15%, limited to code suggestions, CSS formatting, and documentation.
- All final implementation logic, architectural design, debugging, and integration were done manually.

Component / Tool	Purpose / Role in Project	Usage %	Notes
Flask (App + API)	Backend logic, routing, request handling	15%	Strictly used to serve JSON REST APIs rather than rendering full HTML pages.
SQLite + SQLAlchemy	Data modeling, relationships, persistence	12%	Handles complex querying and relationships between students, companies, and drives.
Vue.js (Core)	Reactive frontend, dynamic page rendering	25%	Central to the entire frontend experience (renders components without page reloads).
Fetch API (AJAX)	Handling async API calls from frontend	10%	Powers all communication between Vue and the Flask backend.
Bootstrap / CSS	Styling and responsive layout	8%	Used to create modern, responsive dashboard layouts and forms.
Authentication (JWT)	Login/logout, role-based access control	10%	Uses stateless JWT tokens to verify and distinguish Admin, Student, and Company routes.
Admin CRUD Logic	Manage users, companies, and drives	5%	Dedicated frontend dashboards supported by secure backend APIs.
Background Jobs (Celery/Redis)	Async tasks (emails, PDF reports, exports)	15%	Handles all heavy processing and automated scheduled jobs outside the main Flask thread.

4. Technologies and Frameworks Used

Technology / Library	Purpose

Flask	Core backend web framework and REST API provider
Vue.js 3	Frontend JavaScript framework (via CDN) for dynamic, reactive UI
Bootstrap 5	Frontend HTML generation and responsive styling
SQLAlchemy	Object Relational Mapper (ORM) for database interactions
SQLite	Lightweight local database for storing application data
Redis	In-memory data store used as a message broker and cache backend
Celery	Asynchronous task queue for background jobs and scheduled tasks
Flask-JWT-Extended	User authentication, token generation, and secure session management

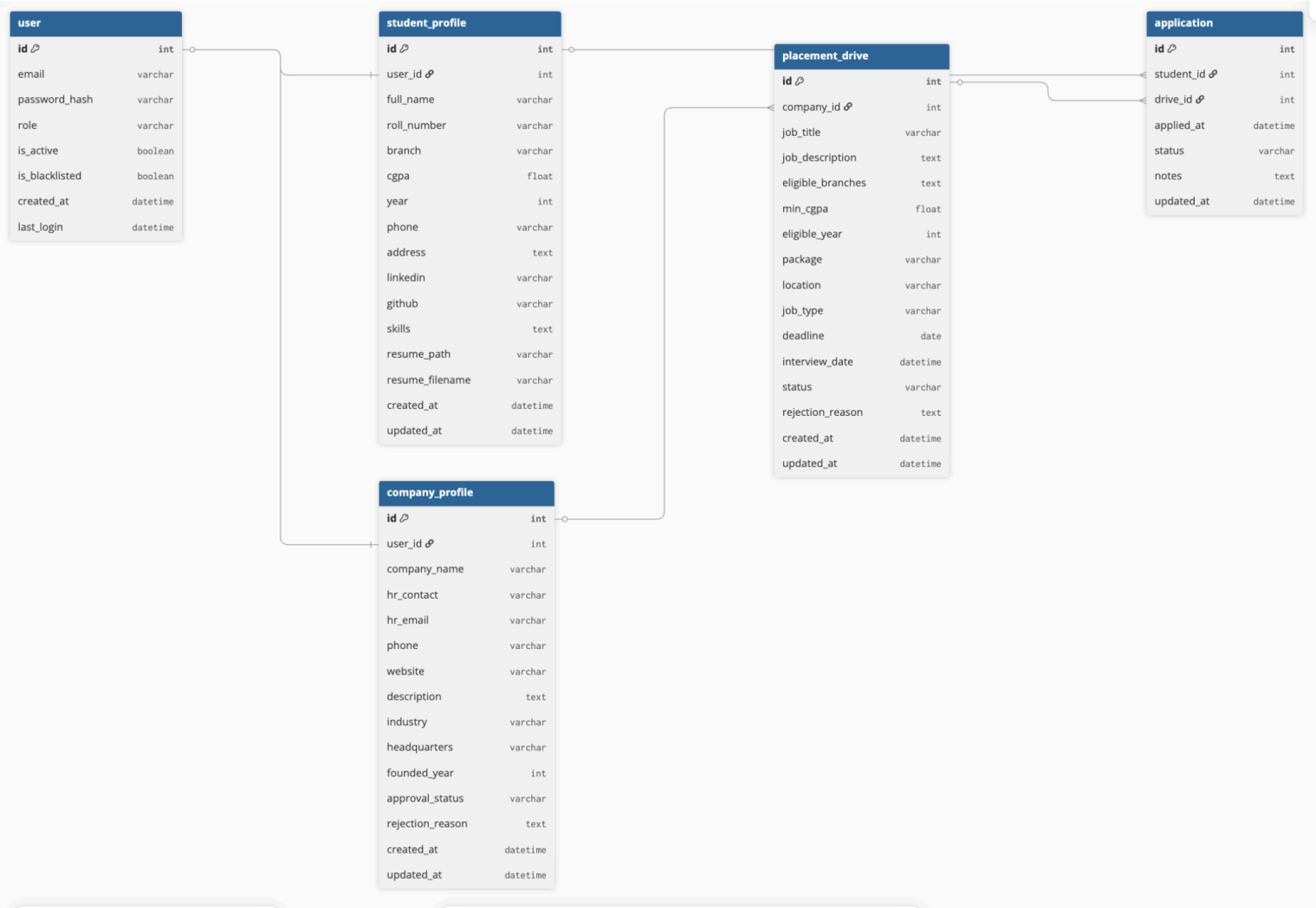
5. Database Schema / ER Diagram

Tables:

- **User** — Stores core authentication details (`id`, `email`, `password_hash`, `role`, `is_active`, `is_blacklisted`).
- **StudentProfile** — Stores student details (`id`, `user_id`, `full_name`, `roll_number`, `branch`, `cgpa`, `phone`).
- **CompanyProfile** — Stores company details (`id`, `user_id`, `company_name`, `industry`, `approval_status`).
- **PlacementDrive** — Logs job postings (`id`, `company_id`, `title`, `description`, `min_cgpa`, `status`).
- **Application** — Tracks student applications (`id`, `student_id`, `drive_id`, `status`, `applied_on`).

Relationships:

- **One-to-One** → `User` ↔ `StudentProfile` / `CompanyProfile`
- **One-to-Many** → `CompanyProfile` → `PlacementDrive`
- **One-to-Many** → `StudentProfile` → `Application`
- **One-to-Many** → `PlacementDrive` → `Application`



6. API Resource Endpoints

Endpoint	Method	Description
`/api/auth/login`	POST	Authenticate user and generate JWT session token
`/api/auth/register`	POST	Register a new student or company profile
`/api/admin/dashboard`	GET	Fetch system-wide statistics (counts of students, companies, etc.)
`/api/company/drives`	POST	Create a new placement drive (Requires company auth)
`/api/student/applications`	POST	Apply to an active placement drive (Requires student auth)

`/api/tasks/export-csv`	POST	Trigger an asynchronous Celery task to export drive data

7. Architecture and Features

Architecture Overview:

- `backend/app.py` – Main Flask application factory and Celery initialization.
- `backend/models/` – Database models using SQLAlchemy.
- `backend/routes/` – Flask Blueprints defining modular REST APIs (`auth`, `admin`, `company`, `student`).
- `backend/tasks/` – Celery background workers (`daily_reminder.py`, `monthly_report.py`, `export_csv.py`).
- `frontend/src/` – Frontend interface using pure HTML, Bootstrap, and Vue.js directly in the browser without a build step.

Implemented Features:

- Unified Role-Based Access Control (Admin, Company, Student).
- Admin dashboard for moderating (approving/rejecting/blacklisting) companies and drives.
- Dynamic eligibility-based filtering for students applying to drives.
- Secure password hashing and JWT Token session handling.
- Responsive and modern UI built entirely with Bootstrap.

Advanced Asynchronous Features (Celery & Redis):

- **User Triggered Job:** Export placement drive data to a CSV file in the background for student in their applications dashboard.
- **Scheduled Job 2 (Monthly):** Generates an HTML/PDF activity report and emails it to the Admin on the 1st of every month.

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1mCoqEXu833E4u1UguO5oEbhOK92P0MMX/view?usp=sharing

(Accessible to all with “View” permission.)